$3n+1$ conjecture: a proof or almost

Alexandre MAKARENKO

April 2, 2022

The Collatz algorithm is rewritten to remove divisions by two and to transform it from a hailstone to a steadily growing value. In contrast with the original problem this new sequence becomes reversible and it is reverted in combinatorial way to find all integers leading to the sequence end. Computer programs are available for demonstrations and experimenting.

1 Collatz differently

1.1 Notations

In the original formulation for any integer number $X_i > 0$ to obtain X_{i+1} we either multiply X_i by 3 and add 1 if it is odd, or divide it by 2 until the result remains even. Such an algorithm leads to a, so called, hailstone behaviour of X_i.

For any integer number $X_i > 0$ represented in base-2 we will use H_i (Head) to designate the most significant bit position and T_i(Tail) the least significant bit position (number of trailing zeros). For example for a binary number

```
10001010101000
H000000000T000
```

$H = 13$ and $T = 3$.

1.2 Key statement

The new sequence will be

$$X_{i+1} = 3X_i + 2^{T_i} \tag{1.1}$$

and Collatz states it will eventually lead to $H_n = T_n$

$$X_n = 2^{H_n} = 2^{T_n} \tag{1.2}$$

In other words to a single 1 shifted left by T_n bits.

T_n is exactly the number of divisions by 2 we would accomplish with the regular Collatz algorithm.

Any additional step for $i > n$ will merely multiply X_i by 4

$$X_{i+1} = 3X_i + 2^{T_i} = 3 \cdot 2^{T_i} + 2^{T_i} = 4 \cdot 2^{T_i} = 2^{T_i+2}$$

or shift it left by two positions.

Example for 49:

i	binary X_i	decimal X_i	original X_i
0	110001	49	49
1	10010100	148	37
2	111000000	448	7
3	10110000000	1408	11
4	1000100000000	4352	17
5	11010000000000	13312	13
6	1010000000000000	40960	5
7	1000000000000000000	131072	(end)1
8	10000000000000000000	524288	(useless)1

Let us demonstrate in details the step 2 for $X_2 = 448 = 111000000$. After multiplication by 3 instead of dividing the result by 2 we add $2^6 = 1000000$:

i	binary X_i	decimal X_i	original X_i
2	111000000	448	7
	10101000000	448*3	
	+1000000	$+2^6$	
3	10110000000	1408	11

With this new formulation the recursion will be:

$$X_1 = 3X_0 + 2^{T_0}$$

$$X_2 = 3X_1 + 2^{T_1} = 3\left(3X_0 + 2^{T_0}\right) + 2^{T_1} = 3^2 \cdot X_0 + 3^1 \cdot 2^{T_0} + 3^0 \cdot 2^{T_1}$$

$$X_n = 3^n \cdot X_0 + 3^{n-1} \cdot 2^{T_0} + \cdots + 3^1 \cdot 2^{T_{n-2}} + 3^0 \cdot 2^{T_{n-1}} \tag{1.3}$$

1.3 Sequence properties

Fact 1. *The number of steps to complete the sequence is exactly the number of odd values in the original Collatz.*

Fact 2. *For a step i the new value is exactly the original Collatz value multiplied by T_i.*

Fact 3. *For any $j > i$:*

$$T_j > T_i \tag{1.4}$$

One could say: the problem is no longer a hailstone.

2 Proof or not ?

For a given sequence end $X_n = 2^{T_n}$ there are generally many starting points X_0 leading to X_n. For instance, both $X_0 = 26$ and $X_0 = 85$ end with $X_n = 256 = 2^8$.

2.1 Key moment

Instead of generating values according to Eq.1.2 we will look for all possible paths back from $2^{T_n} = 1 \ll T_n$.

Reverting Eq.1.2 yields

$$X_i = \left(X_{i+1} - 2^{T_i}\right)/3 \tag{2.1}$$

where

$$0 \leq T_i < T_{i+1} \tag{2.2}$$

and

$$X_{i+1} - 2^{T_i} \equiv 0 \ (mod\, 3) \tag{2.3}$$

This means that starting from a $X_n = 2^{T_n}$ we can find all possible values for X_{n-1} by testing T_{n-1} against Eq.2.2 and Eq.2.3. Then repeat for each T_{n-1}. And so on we will discover all values leading to X_n.

For example, observing closer a value $2^{T_n} = 1000000...0000$ one can see that the number of suitable values for T_{n-1} is $T_n/2$ (number of zero pairs). Moreover, the lowest acceptable $T_{n-1} = 1$ if T_n is even otherwise $T_{n-1} = 2$. While the highest is always $T_{n-1} = T_n - 2$.

All child values of 2^{T_n} with even T_n and odd T_n never overlap (see Example). Thus picking up two large starting points 2^{T_n-1} and 2^{T_n} will seed uniquely values situated below $2^{T_n}/3$.

Example of all values reverted from 2^{10}:

step 0	step 1	step 2	step 3	step 4	step 5	binary
1024						10000000000
	341					101010101
	340					101010100
		113				1110001
	336					101010000
	320					101000000
		106				1101010
			35			100011
		104				1101000
			34			100010
				11		1011
		96				1100000
	256					100000000
		85				1010101
		84				1010100
		80				1010000
			26			11010
			24			11000
		64				1000000
			21			10101
			20			10100
				6		110
			16			10000
				5		101
				4		100
					1	1

Let's name N_i the number of unique values produced by 2^i. Since 2^iproduces all values of 2^{i-2}, N_i will not include values of N_{i-2} and so on. In our example above, $N_{10} = 12$: {1024, 341, 340, 113, 336, 320, 106, 35, 104, 34, 11, 96}, $N_8 = 6$: {256, 85, 84, 80, 26, 24}.

N_i is a deterministic counting function of i which grows as

$$N_i = \frac{4}{3} N_{i-1} \tag{2.4}$$

Running the command "`java Numbers X`" in ./src displays all values for N_X. For example:

```
> java Numbers 14
1 ^14-14 16384
.  5 ^12-10 5120
.  .  3 ^10-9 1536
.  .  13 ^10-7 1664
.  .  .  17 ^9-5 544
.  .  .  .  11 ^7-4 176
.  .  .  .  .  7 ^5-3 56
.  .  .  .  .  .  9 ^4-1 18
.  .  .  .  .  29 ^5-1 58
.  .  .  .  .  .  19 ^4-0 19
.  .  .  .  45 ^7-2 180
.  .  .  .  181 ^7-0 181
.  .  .  69 ^9-3 552
.  .  .  277 ^9-1 554
.  .  53 ^10-5 1696
.  .  .  35 ^9-4 560
.  .  .  .  23 ^7-3 184
.  .  .  .  .  15 ^5-2 60
.  .  .  .  .  61 ^5-0 61
.  .  .  .  93 ^7-1 186
.  .  .  141 ^9-2 564
.  .  .  565 ^9-0 565
.  .  213 ^10-3 1704
.  .  853 ^10-1 1706
.  21 ^12-8 5376
.  85 ^12-6 5440
.  .  113 ^10-4 1808
.  .  .  75 ^9-3 600
.  .  .  301 ^9-1 602
.  .  453 ^10-2 1812
.  .  1813 ^10-0 1813
.  341 ^12-4 5456
.  .  227 ^10-3 1816
.  .  .  151 ^9-2 604
.  .  .  .  201 ^7-0 201
.  .  .  605 ^9-0 605
.  .  909 ^10-1 1818
.  1365 ^12-2 5460
.  5461 ^12-0 5461
total: 39 - new(red): 8 - mul3(yellow): 14
```

```
> java Numbers 15
1 ^15-15 32768
.  5 ^13-11 10240
.  .  3 ^11-10 3072
.  .  13 ^11-8 3328
.  .  .  17 ^10-6 1088
.  .  .  .  11 ^8-5 352
.  .  .  .  .  7 ^6-4 112
.  .  .  .  .  .  9 ^5-2 36
.  .  .  .  .  .  37 ^5-0 37
.  .  .  .  .  29 ^6-2 116
.  .  .  .  .  .  19 ^5-1 38
.  .  .  .  .  117 ^6-0 117
.  .  .  .  45 ^8-3 360
.  .  .  .  181 ^8-1 362
.  .  .  69 ^10-4 1104
.  .  .  277 ^10-2 1108
.  .  .  .  369 ^8-0 369
.  .  .  1109 ^10-0 1109
.  .  53 ^11-6 3392
.  .  .  35 ^10-5 1120
.  .  .  .  23 ^8-4 368
.  .  .  .  .  15 ^6-3 120
.  .  .  .  .  61 ^6-1 122
.  .  .  .  93 ^8-2 372
.  .  .  .  373 ^8-0 373
.  .  .  141 ^10-3 1128
.  .  .  565 ^10-1 1130
.  .  213 ^11-4 3408
.  .  853 ^11-2 3412
.  .  .  1137 ^10-0 1137
.  .  3413 ^11-0 3413
.  21 ^13-9 10752
.  85 ^13-7 10880
.  .  113 ^11-5 3616
.  .  .  75 ^10-4 1200
.  .  .  301 ^10-2 1204
.  .  .  .  401 ^8-0 401
.  .  .  1205 ^10-0 1205
.  .  453 ^11-3 3624
.  .  1813 ^11-1 3626
.  341 ^13-5 10912
.  .  227 ^11-4 3632
.  .  .  151 ^10-3 1208
.  .  .  .  201 ^8-1 402
.  .  .  605 ^10-1 1210
.  .  .  .  403 ^8-0 403
.  .  909 ^11-2 3636
.  .  3637 ^11-0 3637
.  1365 ^13-3 10920
.  5461 ^13-1 10922
total: 50 - new(red): 11 - mul3(yellow): 17
```

where each j-th value X_{ij} produced from 2^i is displayed as

$$. \quad . \quad . \quad v_{ij} \; \hat{} L - R \; X_{ij}$$

meaning $X_{ij} = v_{ij} \times 2^R$ with L and R as leftmost and rightmost bits respectively. L for information only and the dotted indentation is the depth to display better the childhood relationship.

v_{ij} are printed in green or yellow when $v_{ij} \equiv 0 \; (mod\, 3)$. X_{ij} are printed in blue or red when $R = 0$.

N_i, in other words, is the number of all X_{ij}:

$$N_i = |\{X_{ij}\}|$$

Computing $\{v_{i+1,j}\}$ is producing exactly the same values as $\{v_{ij}\}$ since starting from 2^{i+1} provides only one trailing 0 more. Due to this 0, there are two possibilities for an $X_{i+1,j}$ coming from X_{ij}.

Either it is a double of the previous value (blue)

$$X_{i+1,j} = 2X_{ij}$$

or occasionally new value (red)

$$X_{i+1,j} = 2\left(X_{ij} - 1\right)/3$$

This makes all X_{ij} unique.

It is important to remark that

1. Absolutely all $V_i = \{v_{ij}\}$ are naturally found in $V_{i+1} = \{v_{i+1,j}\}$ while none of $\{X_{ij}\}$ will exist in $\{X_{kj}\}$ for $k > i$.
2. The red values are the only new numbers between V_{i+1} and V_i.
3. The yellow values never have children since they are multiple of 3 and will never satisfy Eq.2.3 due to $3K - 2^m \not\equiv 0 \ (mod\, 3)$ for any K and m.

Starting from 2^{i+1} doubles the number of possible values compared to 2^i but only $\frac{2}{3}$ of all numbers are green and will have children, so $N_{i+1} = 2\frac{2}{3}N_i$.

This is also verifiable from computational results. Looking at the Example A, one can notice that for every 8 rows $N_i \simeq 10N_{i-8}$. For instance

$$N_{70} = 373815696$$
$$N_{78} = 3733966810$$

In fact,

$$\left(\frac{4}{3}\right)^8 \simeq 9.9887212$$

and

$$\frac{N_{78}}{N_{70}} \simeq 9.9887908$$

So more precisely $N_i = \left(\frac{4}{3}\right)^8 N_{i-8}$.

Proof. By tending n to infinity, the backward sequences from 2^2 and 2^{n-1} will produce all integers from 1 to ∞. If any other cycle (not ending by 1) existed, it would lead to an infinite number of values breaking the counting function N_i. □

3 Example A

N_i up to $i = 80$:

i	N_i	$\sum N_i$	$(N_i - N_{i-1})/N_{i-1}$		i	N_i	$\sum N_i$	$(N_i - N_{i-1})/N_{i-1}$
0	1	1	0.000000		41	89009	356033	0.333209
1	1	2	0.000000		42	118631	474664	0.332798
2	1	3	0.000000		43	158171	632835	0.333302
3	1	4	0.000000		44	210939	843774	0.333614
4	2	6	1.000000		45	281334	1125108	0.333722
5	3	9	0.500000		46	375129	1500237	0.333394
6	4	13	0.333333		47	500106	2000343	0.333157
7	5	18	0.250000		48	666725	2667068	0.333167
8	6	24	0.200000		49	888947	3556015	0.333304
9	8	32	0.333333		50	1185305	4741320	0.333381
10	12	44	0.500000		51	1580518	6321838	0.333427
11	18	62	0.500000		52	2107346	8429184	0.333326
12	24	86	0.333333		53	2809845	11239029	0.333357
13	31	117	0.291667		54	3746399	14985428	0.333312
14	39	156	0.258065		55	4995078	19980506	0.333301
15	50	206	0.282051		56	6660211	26640717	0.333355
16	68	274	0.360000		57	8880688	35521405	0.333394
17	91	365	0.338235		58	11840592	47361997	0.333297
18	120	485	0.318681		59	15787976	63149973	0.333377
19	159	644	0.325000		60	21050985	84200958	0.333356
20	211	855	0.327044		61	28067940	112268898	0.333331
21	282	1137	0.336493		62	37423702	149692600	0.333326
22	381	1518	0.351064		63	49897977	199590577	0.333326
23	505	2023	0.325459		64	66531372	266121949	0.333348
24	665	2688	0.316832		65	88710360	354832309	0.333361
25	885	3573	0.330827		66	118280689	473112998	0.333336
26	1187	4760	0.341243		67	157705535	630818533	0.333316
27	1590	6350	0.339511		68	210272571	841091104	0.333324
28	2122	8472	0.334591		69	280362436	1121453540	0.333329
29	2829	11301	0.333176		70	373815696	1495269236	0.333330
30	3765	15066	0.330859		71	498423661	1993692897	0.333341
31	5014	20080	0.331740		72	664571765	2658264662	0.333347
32	6682	26762	0.332669		73	886092554	3544357216	0.333329
33	8902	35664	0.332236		74	1181447507	4725804723	0.333323
34	11878	47542	0.334307		75	1575258367	6301063090	0.333329
35	15844	63386	0.333895		76	2100349800	8401412890	0.333337
36	21122	84508	0.333123		77	2800476278	11201889168	0.333338
37	28150	112658	0.332734		78	3733966810	14935855978	0.333333
38	37536	150194	0.333428		79	4978612219	19914468197	0.333331
39	50067	200261	0.333840		80	6638155867	26552624064	0.333335
40	66763	267024	0.333473					

4 Example B

Values reverted from 2^7 and 2^8 with Eq.2.1:

```
128          10000000  odd  Tn=7
  42           101010  = (10000000-10)/11          = 1111110/11
  40           101000  = (10000000-1000)/11        = 1111000/11
    13           1101  = (101000-1)/11             = 100111/11
    12           1100  = (101000-100)/11           = 100100/11
  32           100000  = (10000000-100000)/11      = 1100000/11
    10           1010  = (100000-10)/11            = 11110/11
      3            11  = (1010-1)/11               = 1001/11
    8            1000  = (100000-1000)/11          = 11000/11
      2            10  = (1000-10)/11              = 110/11

256         100000000  even  Tn=8
  85          1010101  = (100000000-1)/11          = 11111111/11
  84          1010100  = (100000000-100)/11        = 11111100/11
  80          1010000  = (100000000-10000)/11      = 11110000/11
    26          11010  = (1010000-10)/11           = 1001110/11
    24          11000  = (1010000-1000)/11         = 1001000/11
  64          1000000  = (100000000-1000000)/11    = 11000000/11
    21          10101  = (1000000-1)/11            = 111111/11
    20          10100  = (1000000-100)/11          = 111100/11
      6            110  = (10100-10)/11            = 10010/11
    16          10000  = (1000000-10000)/11        = 110000/11
      5            101  = (10000-1)/11             = 1111/11
      4            100  = (10000-100)/11           = 1100/11
        1            1  = (100-1)/11               = 11/11
```

5 Source code

This document and computer programs may be found here:

https://github.com/sashamakarenko/collatz